

Online Test Platformu PRD ve Yol Haritasi

İçindekiler

1. Proje Özeti
2. Ekip ve Sorumluluk Yaklaşımı
3. Ürün Hedefleri
4. Kullanıcı Roller
5. Temel Ekranlar
6. Netleştirilmiş Teknik Yapı
7. Netleştirilmiş Veritabanı Modelleri
8. Fazlara Bölünmüş Yol Haritasi
9. Arda İçin Net Görev Sahipliği
10. Emir İçin Net Görev Sahipliği
11. Önerilen Haftalık Çalışma Planı
12. MVP Kapsamı (Sayfa ve Akis Bazında)
13. Verilmiş Kararlar
14. Başarı Kriterleri
15. Dosya ve Kod Organizasyonu Kuralları
16. İlk Yapılacaklar
17. Araştırma Kaynakları
18. Gizlilik, KVKK ve Hukuki Notlar
19. Hata Yönetimi, Loglama ve İzleme
20. SEO ve Metadata
21. Erişilebilirlik ve Tarayıcı Desteği
22. Test Güvenliği
23. İçerik Sayfaları
24. Backend Alternatifleri (Araştırma Özeti)

1. Proje Özeti

Bu proje, öğrencilerin ders seçerek online test çözebildiği, hocaların ise soru havuzu, test yönetimi ve öğrenci takibi yapabildiği bir online sınav platformudur.

Platform iki ana kullanıcı grubuna hizmet eder:

- Öğrenciler: Ana sayfadan online teste girer, ders seçer, test çözer ve sonucunu görür.
- Hocalar: Üyelik/giriş sistemiyle panele girer, kendisine tanımlı öğrencileri görür, soru yükler, soru havuzundan test oluşturur ve sınava giren öğrencileri takip eder.

Projede ayrıca reklam alanları bulunacaktır. Reklam bölümleri ilk etapta statik alanlar olarak başlayacak, sonraki fazlarda panelden yönetilebilir hale getirilecektir.

Bu doküman projenin ana karar kaynağıdır. Uygulamada yapılacak değişiklikler önce bu dosyaya eklenmeli, sonra kodlanmalıdır.

2. Ekip ve Sorumluluk Yaklaşımı

Projeyi iki kişi geliştirecek:

- Arda
- Emir

İş bölümü alan sahipliğine göre ayrılır. Her kişi kendi alanındaki ekran, component veya teknik akisin tamamlanmasından sorumludur.

Arda'nın ana sorumluluk alanı:

- Proje kurulumu ve mimari kararlar

- Veritabani tasarimi
- Auth ve rol kontrolu
- Ogrenci test akisinin is mantigi
- Test sonuc hesaplama ve kayit
- Hoca panelindeki veri akislari
- Soru havuzu ve test olusturma backend mantigi
- Deployment ve teknik kontroller

Emir'in ana sorumluluk alanı:

- Ana sayfa arayuzu
- Header, footer ve ortak gorsel bolumler
- Ders kartlari ve ders secim sayfasi arayuzu
- Ogrenci bilgi formu arayuzu
- Test baslangic ve test cozum ekranlarinin gorsel duzeni
- Sonuc ekrani arayuzu
- Hoca paneli dashboard kartlari
- Liste, tablo ve bos durum ekranlari
- Reklam alanlari
- Demo icerik ve sayfa metinleri

Ortak calisma kurallari:

- Her faz sonunda calisan bir ekran veya akis teslim edilir.
- Arda teknik entegrasyonu ve veri baglantilarini kontrol eder.
- Emir arayuz, icerik ve responsive gorunumu kontrol eder.
- Bir gorev baska kisinin alanini etkiliyorsa degisiklik once bu dokumana eklenir.

3. Urun Hedefleri

Ana Hedefler

- Ogrencilerin kolayca ders secip test cozmesini saglamak.
- Hocalarin soru yukleyip soru havuzundan test olusturabilmesini saglamak.
- Hocalarin sinava giren ogrencileri ve sonuclarini gorebilmesini saglamak.
- Hocalara kendisine tanimli ogrenciler uzerinden takip imkani vermek.
- Reklam alanlariyla platformda gelir modeli icin temel hazirlamak.

Ilk Surumde Odak

Ilk surumda amac, urunun calisan bir MVP halini cikarmaktır:

- Ana sayfa
- Ogrenci icin ders secimi
- Test cozme ekrani
- Test sonucu kaydi
- Hoca girişi
- Hoca paneli
- Soru havuzu
- Test olusturma
- Sinava giren ogrencileri goruntuleme
- Temel reklam alanlari

4. Kullanici Roller

4.1 Ziyaretci / Ogrenci

Ogrenci ilk etapta uyelik olmadan teste girebilir.

Ogrenci akisi:

1. Ana sayfaya girer.
2. "Online Test" butonuna basar.
3. Dersleri gorur.
4. Bir ders secer.
5. O derse ait aktif testleri gorur.
6. Bir test secer.
7. Ogrenci bilgi formunu doldurur ve e-posta OTP dogrulasini tamamlar.
8. Testi cozer.
9. Testi bitirir.
10. Sonucunu gorur.

Ogrenciden alinabilecek bilgiler:

- Ad soyad
- E-posta (zorunlu)
- Telefon (opsiyonel)
- Sinif seviyesi
- Bagli oldugu hoca, varsa

Not: "Herhangi biri girebilecek" istegi nedeniyle ogrenci uyelig zorunlu olmayacaktır. Ancak sonuc takibi ve tekrar deneme kontrolu icin test baslamadan once dogrulanmis e-posta alinmalidir.

Kimlik dogrulama:

- Ogrenci bilgi formunda girilen e-posta adresine tek kullanimlik dogrulama kodu (OTP) gönderilir.
- OTP dogrulanmadan test baslatilamaz.
- Bu adim sahte bilgi girilerek tekrar deneme engelinin bypass edilmesini onler.
- OTP göndermek icin e-posta servisi (Resend) kullanılacaktır. SMS entegrasyonu ileri faza birakilmistir.
- OTP kodlari 5 dakika gecerlidir ve tek kullanimlidir.
- Telefon MVP'de sadece iletisim bilgisi olarak tutulur; kimlik dogrulama ve tekrar deneme kontrolunde kullanilmaz.

4.2 Hoca

Hoca uyelik/giris sistemiyle panele girer.

Hoca yetenekleri:

- Kendi paneline giris yapar.
- Kendisine tanimli ogrencileri gorur.
- Sinava giren ogrencileri gorur.
- Ogrenci sonuclarini inceler.
- Soru yukler.
- Soru havuzunu gorur.
- Soru havuzundan test olusturur.
- Testleri aktif/pasif yapar.

4.3 Admin

Admin rolu ilk etapta temel kapsamda tutulabilir ama sistemin saglikli ilerlemesi icin gereklidir.

Admin yetenekleri:

- Hoca hesaplarini yonetir.
- Hocalara ogrenci tanimlar.
- Dersleri yonetir.

- Reklam alanlarını ilerleyen fazlarda yönetir.
- Gerekirse tüm soru havuzunu ve testleri denetler.

5. Temel Ekranlar

5.1 Ana Sayfa

İçerik:

- Platform tanıtımı
- "Online Teste Basla" butonu
- Derslere yönlendirme
- Hocalar için giriş butonu
- Reklam alanı
- Temel bilgilendirme bölümleri

Sorumluluk:

- Emir: Tasarım, statik içerik, reklam alanlarının yerleştirilmesi.
- Arda: Online test ve hoca paneli yönlendirme mantığı.

5.2 Ders Seçim Sayfası

İçerik:

- Ders kartları
- Her ders için ad, açıklama ve test sayısı
- Aktif testi olmayan derslerde uygun bilgilendirme

Sorumluluk:

- Emir: Ders kartları ve sayfa düzeni.
- Arda: Ders verilerinin veritabanından çekilmesi.

5.3 Test Başlangıç Sayfası

İçerik:

- Seçilen dersin adı
- Test kuralları
- Öğrenci bilgi formu
- "Teste Basla" butonu

Sorumluluk:

- Emir: Form arayuzu ve açıklama metinleri.
- Arda: Form verisinin doğrulanması ve test oturumu başlatma.

5.4 Test Çözme Ekranı

İçerik:

- Soru metni
- Şıklar
- Sonraki / önceki soru
- Cevap işaretleme
- Sure varsa sure göstergesi
- Testi bitir butonu

Sorumluluk:

- Arda: Test motoru, cevap kaydı, süre, sonuç hesaplama.
- Emir: Soru kartı arayuzu ve görsel iyileştirmeler.

5.5 Sonuc Ekrani

Icerik:

- Dogru sayisi
- Yanlis sayisi
- Bos sayisi
- Puan
- Ders bilgisi
- Soru bazli cevap ozeti (ogrencinin secimi ve dogru cevap)

Sorumluluk:

- Arda: Sonuc hesaplama ve kaydetme.
- Emir: Sonuc ekraninin tasarimi.

5.6 Hoca Girisi

Icerik:

- E-posta ve sifre giris formu
- Hatali giris uyarisi
- Sifre sifirlama akisi (Faz 9'da eklenecek)

Sorumluluk:

- Arda: Auth, rol kontrolu, guvenlik.
- Emir: Form tasarimi.

Notlar:

- Hocalar kendileri kayit olamaz; hesaplari admin tanimlar (bkz. Bolum 13).
- Public bir kayit sayfasi MVP'de olmayacaktır.

5.7 Hoca Paneli

Icerik:

- Ozet dashboard
- Toplam ogrenci
- Sinava giren ogrenciler
- Son test sonuclari
- Hizli aksiyonlar

Sorumluluk:

- Arda: Panel veri mantigi.
- Emir: Dashboard kartlari ve temel gorsel duzen.

5.8 Soru Havuzu

Icerik:

- Sorulari listeleme
- Derse gore filtreleme
- Zorluk seviyesine gore filtreleme
- Soru ekleme
- Soru duzenleme
- Soru pasife alma (soft-delete)

Sorumluluk:

- Arda: Veritabani, CRUD, validasyon, yetki kontrolu.
- Emir: Liste arayuzu, filtre alanlari ve form tasarimi.

Notlar:

- Veri butunlugu icin sorular kalici olarak silinmez.
- `is_active = false` ile pasife alinir.
- Bir teste ekli sorular pasife alinsa bile testlerde gorunmeye devam eder; sadece havuzda yeni eklemelerde listelenmez.

5.9 Test Olusturma

Icerik:

- Test adi
- Ders secimi
- Soru havuzundan soru secimi
- Testi aktif/pasif yapma
- Test ayarlari (sure, uyeliksiz katilim, sonuc gosterimi)

Notlar:

- Hoca yalnızca kendi sorularini teste ekleyebilir (bkz. Bolum 13). Ortak soru havuzu ileri fazda dusunulebilir.

Sorumluluk:

- Arda: Test olusturma mantigi, soru-test iliskisi, aktif test kontrolu.
- Emir: Form tasarimi ve secim arayuzu.

5.10 Sinava Giren Ogrenciler

Bu sayfa, **hocanın kendi olusturdugu testlere giren** ogrencileri listeler. Her hoca yalnızca kendi testlerine ait sonuclari gorur. Tum sistem genelinde sinava giren ogrencileri sadece admin gorebilir.

Icerik:

- Ogrenci adi
- Test adi
- Ders
- Puan
- Dogru/yanlis/bos sayilari
- Sinav tarihi
- Detay görüntüleme

Sorumluluk:

- Arda: Liste sorgulari, filtreleme, yetki kontrolu.
- Emir: Tablo/list UI ve bos durum ekranlari.

5.11 Reklam Bolumu

Ilk faz:

- Ana sayfada reklam alani
- Ders secim sayfasinda reklam alani
- Sonuc ekraninda reklam alani

MVP reklam standardi:

- Reklamlar statik placeholder component olarak baslar.
- Ana sayfa reklam alani yatay banner olarak konumlanir.
- Ders secim sayfasinda kart grid'ini bozmayacak yatay veya dikey placeholder kullanilir.
- Sonuc ekraninda sonuc ozetinden sonra yatay banner kullanilir.
- Reklam componentleri mobilde tam genislik, desktop'ta max-width sinirli calisir.
- Tiklanma takibi MVP'de yoktur.

Ileri faz:

- Admin panelden reklam ekleme
- Reklam aktif/pasif yapma
- Baslangic ve bitis tarihi
- Tiklanma takipleri

Sorumluluk:

- Emir: Statik reklam alanlari ve gorsel yerlestirme.
- Arda: Ileri fazdaki reklam yonetimi ve veritabani mantigi.

6. Netlestirilmis Teknik Yapı

Internet arastirmasi ve proje kapsamindan sonra MVP icin tek ana teknoloji yolu secilmistir. Bu projede ayri bir backend uygulaması kurulmayacak; frontend, server-side islemler, API ihtiyaclari, auth entegrasyonu ve veritabani erisimi Next.js icinde yonetilecektir.

6.1 Ana Teknoloji Stack'i

- Framework: Next.js App Router
- Dil: TypeScript
- UI: Tailwind CSS + shadcn/ui
- Form ve validasyon: React Hook Form + Zod
- Tablo/listeleme: TanStack Table
- ORM: Prisma
- Veritabani: PostgreSQL
- Veritabani hosting: Neon
- Auth: Auth.js (NextAuth)
- Yetkilendirme: Uygulama seviyesinde RBAC + server-side ownership kontrolu
- Dosya/gorsel saklama: Vercel Blob, ilerleyen fazda
- Deployment: Vercel
- Versiyon kontrol: Git + GitHub

6.2 Neden Bu Stack?

- Next.js App Router, public sayfalar, ogrenci test akisi, hoca paneli ve admin panelini ayni proje icinde duzenli ayirmaya uygundur.
- Prisma, PostgreSQL ile tip guvenli veritabani erisimi ve migration yonetimi saglar.
- PostgreSQL, hoca-ogrenci-test-sonuc gibi iliskisel veri yapilari icin uygundur.
- Auth.js (NextAuth), Next.js icinde hoca/admin girisi, session yonetimi ve rol kontrolu icin kullanilir.
- shadcn/ui component kodunu proje icine aldigi icin dashboard, form, tablo ve dialog gibi parcalar kolayca ozellestirilebilir.
- Vercel, Next.js projeleri icin hizli preview ve production deployment saglar.

6.3 Next.js Mimari Kararlari

Uygulama `src` klasoru altinda gelistirilecektir.

Onerilen route gruplari:

- `src/app/(public)` : Ana sayfa, ders secim sayfasi, reklamli public alanlar.
- `src/app/(student)` : Test baslangic, test cozme ve sonuc ekranlari.
- `src/app/(teacher)` : Hoca girisi sonrasi panel sayfali.
- `src/app/(admin)` : Admin tarafından kullanılacak yonetim sayfali.
- `src/app/api` : Gerekli route handler endpointleri.

Temel route yapisi:

- `/` : Ana sayfa
- `/online-test` : Ders secim sayfasi
- `/online-test/[courseId]` : Dersin aktif test listesi
- `/test/[testId]/start` : Ogrenci bilgi formu ve test kurallari
- `/test/[attemptId]` : Test cozme ekrani
- `/test/[attemptId]/result` : Sonuc ekrani
- `/teacher/login` : Hoca girisi
- `/teacher/dashboard` : Hoca panel ana sayfasi
- `/teacher/questions` : Soru havuzu
- `/teacher/tests` : Test yonetimi

- `/teacher/students` : Kendisine tanımlı öğrenciler
- `/teacher/results` : Sınava giren öğrenciler ve sonuçlar
- `/admin` : Admin paneli, ileri faz

6.4 Backend ve Veritabanı Kararları

Backend tarafında:

- Veritabanı erişimi Prisma üzerinden yapılır.
- Veritabanı şeması `prisma/schema.prisma` dosyasında tutulur.
- Migration'lar Prisma Migrate ile yönetilir.
- Kullanıcı, rol, session ve hesap tabloları `Auth.js` Prisma adaptörü ile yönetilir.
- Uygulama kullanıcı detayları `profiles` tablosunda tutulur.
- Hoca ve admin rolleri `profiles.role` alanıyla başlatılır.
- Hoca paneli verilerinde `teacher_id`, `owner_teacher_id` ve `teacher_students` eşleştirme tablosu üzerinden ownership kontrolü yapılır.
- Client tarafından doğrudan veritabanı erişimi olmayacak; veritabanı işlemleri server action veya route handler içinden yapılacaktır.

Server Action ve Route Handler kullanım kuralı:

- Mutations (veri oluşturma, güncelleme, silme): Server Actions kullanılır.
- Ucuncu parti webhook veya dışarıdan gelen HTTP istekleri: Route Handlers kullanılır.
- Veri okuma (read): Server component içinde doğrudan Prisma sorgusu veya Server Action kullanılır.

Connection pooling:

- Neon veritabanına bağlanırken `-pooler` suffix'li connection string kullanılacaktır.
- `@prisma/adapters-neon` paketi kurularak Vercel serverless ortamında her invocation'da yeni bağlantı açılması onlenecektir.
- `DATABASE_URL` değişkeni pooler endpoint'ini, `DIRECT_URL` değişkeni doğrudan bağlantı endpoint'ini gösterecektir.
- Prisma schema'da `directUrl` ayarı migration işlemleri için kullanılacaktır.

İlk MVP için roller:

- `admin`
- `teacher`

Öğrenci ilk etapta zorunlu üyelik olmadan test cozer. Öğrencinin girdiği bilgiler `students` tablosuna kaydedilir ve test denemesiyle ilişkilendirilir. Öğrenci kimliği MVP'de doğrulanmış e-posta üzerinden tekilleştirilir. Telefon opsiyoneldir ve kimlik doğrulama için kullanılmaz.

Admin paneli MVP kapsamına minimal olarak dahildir. Bu panelde sadece hoca hesabi yönetimi, ders yönetimi ve hoca-öğrenci eşleştirme işlemleri yer alır. Reklam yönetimi, sistem ayarları ve gelişmiş denetim ekranları ileri faza bırakılır.

Hoca ve admin giriş yöntemi:

- MVP'de `Auth.js` Credentials Provider kullanılacaktır.
- Şifreler `bcrypt` ile hashlenerek saklanacaktır.
- Hoca hesapları public kayıt sayfası olmadan admin tarafından oluşturulacaktır.
- İlk admin kullanıcısı seed script ile oluşturulacaktır.
- Google login ve magic link girişi MVP'de yoktur, ileri fazda değerlendirilebilir.

6.5 Güvenlik ve Yetkilendirme

Güvenlik kuralları:

- Hoca paneli giriş yapmayan kullanıcıya kapalı olacak.
- Hoca sadece kendi oluşturduğu testleri ve soruları yönetecek.
- Hoca sadece kendisine tanımlı öğrencileri ve kendi testlerine ait sonuç verilerini görecektir.
- Admin tüm hocaları, dersleri ve eşleştirmeleri yönetebilecek.
- Yetki kontrolü server-side fonksiyonlarda merkezi helper'lar üzerinden yapılacaktır.
- Client tarafından gelen `teacher_id`, `owner_teacher_id` gibi alanlara güvenilmeyecek; aktif session kullanıcısından türetilecek.
- `NEXT_PUBLIC` ile başlayan ortam değişkenleri sadece public değerler için kullanılacaktır.
- `DATABASE_URL`, `auth secret` ve diğer gizli anahtarlar `NEXT_PUBLIC` prefix'i almadan Vercel environment variable olarak saklanacaktır.

6.6 UI ve Tasarim Sistemi

UI kararları:

- Tailwind CSS ile responsive tasarim yapilacak.
- shadcn/ui ile button, card, input, dialog, dropdown, badge, table, sidebar ve form bileşenleri kullanılacak.
- Hoca panelinde sidebar + topbar yapisi olacak.
- Public ogrenci ekranlari sade, hizli ve mobil uyumlu olacak.
- Test ekraninda dikkat dagitmayacak temiz bir tasarim tercih edilecek.

Tasarim dili MVP kararları:

- Font: Inter veya Geist Sans.
- Ana renk: koyu lacivert/indigo tonlari.
- Yardimci renk: yesil veya turkuaz aksan.
- Arka plan: acik gri/beyaz agirlikli sade layout.
- Ikon seti: Lucide React.
- Public sayfalarda mobil-first tasarim uygulanir.
- Hoca panelinde bilgi yogunlugu yuksek ama sade dashboard dili kullanilir.

UI durum yonetimi:

- Her veri cekme ekraninda uc durum tasarlanacaktir: yukleniyor (skeleton/spinner), bos durum (icerik yok mesaji ve yonlendirme) ve hata durumu (kullanici dostu Turkce mesaj ve tekrar dene butonu).
- Yukleniyor durumunda sayfa layout'u korunacak, skeleton componentleri kullanılacaktir.
- Bos durum ekranlarinda kullaniciya ne yapabilecegini anlatan bir mesaj ve aksiyon butonu gosterilecektir.
- Hata durumunda teknik detay gosterilmeyecek; "Bir sorun olustu, lutfen tekrar deneyin." mesaji ve tekrar dene butonu olacaktır.

6.7 Form ve Veri Dogrulama

Formlarda:

- React Hook Form kullanılacak.
- Zod ile schema bazli validasyon yapilacak.
- Ogrenci bilgi formu, soru ekleme formu, test olusturma formu ve giris formlari ortak validasyon mantigiyla yazilacak.
- Hata mesajlari kullaniciya Turkce ve net gosterilecek.

6.8 Kod Organizasyonu ve Servis Katmani

Feature bazli kodlarda ayni kalip izlenecektir:

- `components` : Sadece UI bileşenleri.
- `actions.ts` : Form mutation'lari ve server action giris noktali.
- `queries.ts` : Server-side read sorgulari.
- `schemas.ts` : Zod validasyon semalari.
- `services.ts` : Is kurallari, transaction'lar ve tekrar kullanilan server-side mantik.
- `types.ts` : Feature'a ozel TypeScript tipleri.

Yetki helper'lari `src/lib/auth` veya `src/lib/authorization` altinda tutulur. `requireTeacher()`, `requireAdmin()`, `assertOwnsTest()` ve benzeri fonksiyonlar tum server action ve query'lerde ortak kullanilir.

6.9 Test Stratejisi

MVP icin test yaklasimi:

- Unit test araci olarak Vitest kullanılacaktır.
- Kritik hesaplama fonksiyonlari icin unit test yazilacak.
- Test sonuc hesaplama, bos/dogru/yanlis sayimi ve puanlama mantigi ayrica test edilecek.
- Ogrenci test akisi ve hoca girisi manuel senaryo testleriyle kontrol edilecek.
- Ileri fazda Playwright ile uctan uca testler eklenebilir.

6.10 Deployment Yaklasimi

Deployment Vercel üzerinden yapılacak.

Ortamlar:

- Development: Lokal gelistirme
- Preview: Pull request veya branch bazli Vercel preview
- Production: Canli yayın

Environment variable kurallari:

- `DATABASE_URL` : Sadece server-side veya migration ihtiyacinda kullanilir.
- `AUTH_SECRET` : Auth session imzalama icin kullanilir, gizli tutulur.
- `AUTH_URL` : Deployment URL'sine gore ayarlanir.
- `GOOGLE_CLIENT_ID` : Google login eklenirse kullanilir.
- `GOOGLE_CLIENT_SECRET` : Google login eklenirse server-side gizli tutulur.
- `BLOB_READ_WRITE_TOKEN` : Ileri fazda Vercel Blob ile dosya/gorsel yukleme icin kullanilir.
- `UPSTASH_REDIS_REST_URL` : Rate limiting icin kullanilir.
- `UPSTASH_REDIS_REST_TOKEN` : Rate limiting icin gizli tutulur.
- `RESEND_API_KEY` : OTP, sifre sifirlama ve e-posta bildirimleri icin kullanilir.

Gizli anahtarlar Git'e commit edilmeyecek. Vercel dashboard veya Vercel CLI üzerinden ortamlara ayri ayri tanimlanacak.

CI/CD kontrolleri:

- Pull request veya main branch deploy oncesinde `lint`, `typecheck`, `test` ve `build` komutlari calismalidir.
- Vercel preview deployment basarisizsa merge yapilmaz.
- Production deploy oncesinde migration'lar ve environment variable listesi kontrol edilir.

6.11 Yedekleme ve Geri Yukleme

Yedekleme stratejisi:

- Neon otomatik olarak point-in-time recovery destegi sunar; bu ozellik aktif tutulacaktır.
- Production veritabaninin haftalik manuel yedegi Neon dashboard üzerinden kontrol edilecektir.
- Kritik migration islemlerinden once veritabaninin snapshot'i alinacaktır.
- Geri yukleme senaryosu en az bir kez test edilecektir.

6.12 Ozellik Karari

Online sinav sistemlerinde yaygin gorulen ozelliklerden hangilerinin MVP'de olacagi, hangilerinin ileri faza birakilacagi asagida netlestirilmistir. Sayfa ve akis bazinda MVP kapsamı icin Bolum 12'ye bakilmalidir.

MVP'de uygulanacak ozellikler:

- Soru havuzu (hoca bazli)
- Ders bazli test
- Zorluk seviyesi alanı
- Sureli veya suresiz test yapisi (test bazinda secilebilir)
- Otomatik sonuc hesaplama (server-side)
- Hoca panelinde sonuc listesi
- Ogrenci cevap detaylari (sonuc ekraninda gosterilir)
- Statik reklam alanlari

Ileri faza birakilacak ozellikler:

- Sik karistirma
- Soru sirasi karistirma
- Cevaplar icin auto-save
- PDF sonuc raporu
- Gelismis analitik grafikler
- Coklu soru tipi (cok cevapli, dogru/yanlis, esleme, vb.)
- Ortak soru havuzu
- Reklam ag entegrasyonu (AdSense vb.)

- Sertifika/madalya verme

7. Netlestirilmis Veritabani Modelleri

Veritabani PostgreSQL üzerinde kurulacaktır ve Prisma ORM ile yönetilecektir. Auth.js Prisma adaptoru `User`, `Account`, `Session` ve `VerificationToken` tablolarini otomatik olusturur. Uygulamaya ozel hoca/admin detaylari ayrica `profiles` tablosunda tutulacaktır.

7.1 profiles

Hoca ve admin kullanicilarinin uygulama profil bilgilerini tutar.

Alanlar:

- id
- user_id
- full_name
- email
- password_hash
- role: `admin` veya `teacher`
- avatar_url
- is_active
- created_at
- updated_at

Notlar:

- Hoca girisi Auth.js (NextAuth) ile yapilir.
- `user_id`, auth sistemindeki kullanci kaydina baglanir.
- `password_hash`, Credentials Provider ile giris icin `bcrypt` hash degerini tutar.
- Yetki kontrolunde `profiles.role` ve server-side ownership kontrolleri kullanilir.
- Admin kullanicilar ilk etapta manuel olarak tanimlanabilir.

7.2 students

Teste giren ogrencilerin bilgilerini tutar.

Alanlar:

- id
- full_name
- email (zorunlu, unique)
- phone
- grade_level
- school_name
- created_at
- updated_at

Notlar:

- Ogrenci ilk etapta uyelik acmadan test cozer.
- Ayni ogrencinin tekrar test cozebilmesi icin MVP'de dogrulanmis e-posta ile eslestirme yapilir.
- Ileri fazda ogrenci uyelig eklenirse bu tablo auth kullanicisiyla iliskilendirilebilir.
- `email` alani uzerinde `UNIQUE` constraint uygulanacaktır. Ayni e-posta ile birden fazla kayit olusturulamaz.
- `phone` alani MVP'de opsiyoneldir ve unique constraint almaz.
- Yeni test girisinde mevcut ogrenci kaydi varsa guncellenir, yoksa yeni kayit olusturulur (upsert).

7.2.1 otp_verifications

Ogrenci e-posta dogrulama kodlarini tutar.

Alanlar:

- id

- email
- code_hash
- purpose: student_test_start , password_reset
- expires_at
- consumed_at
- attempt_count
- created_at

Notlar:

- OTP kodlari duz metin olarak saklanmaz; hashlenmis sekilde tutulur.
- OTP 5 dakika gecerlidir.
- Ayni e-posta icin 5 dakikada en fazla 3 OTP gonderilebilir.
- OTP dogrulamada en fazla 5 hatali deneme kabul edilir; sonrasinda yeni kod istenir.
- Kullanilan OTP icin consumed_at doldurulur ve ayni kod tekrar kullanilamaz.

7.3 teacher_students

Hangi ogrencinin hangi hocaya tanimli oldugunu tutar.

Alanlar:

- id
- teacher_id
- student_id
- assigned_by
- assigned_at
- note

Notlar:

- teacher_id , profiles.id alanina baglanir.
- student_id , students.id alanina baglanir.
- Admin, hoca-ogrenci eslestirmesini bu tablo uzerinden yapar.
- Bu yap, ileride bir ogrencinin birden fazla hocaya baglanmasina da izin verir.

7.4 courses

Dersleri tutar.

Alanlar:

- id
- title
- slug
- description
- icon_name
- display_order
- is_active
- created_at
- updated_at

Notlar:

- Dersler public tarafta listelenir.
- Pasif dersler ogrenci tarafinda gosterilmez.

7.5 questions

Soru havuzunu tutar.

Alanlar:

- id
- course_id

- owner_teacher_id
- question_text
- option_a
- option_b
- option_c
- option_d
- correct_option
- difficulty: `easy`, `medium`, `hard`
- topic
- explanation
- image_url
- is_active
- created_at
- updated_at

Notlar:

- MVP'de soru tipi caktan secmeli ve tek dogru cevapli olacak.
- MVP'de her soru 4 siklidir ve `option_a`, `option_b`, `option_c`, `option_d` alanlari zorunludur.
- `correct_option` yalnızca `A`, `B`, `C`, `D` degerlerinden biri olabilir.
- `topic`alani konu bazli filtreleme icin kullanilir.
- `explanation`alani MVP'de hoca panelinde gorunur; ogrenci sonuc ekraninda gosterilmez. Ogrenciye aciklama gosterme ileri faza birakilir.
- `image_url`alani MVP'de kullanilmaz; soru gorseli ve dosya yukleme Vercel Blob entegrasyonu ile ileri fazda eklenir.
- Ortak soru havuzu istenirse ilerleyen fazda `visibility` veya `is_shared`alani eklenebilir.

7.6 tests

Hocalarin olusturdugu testleri tutar.

Alanlar:

- id
- course_id
- owner_teacher_id
- title
- description
- duration_minutes
- status: `draft`, `active`, `archived`
- show_result_immediately
- requires_student_account
- starts_at
- ends_at
- created_at
- updated_at

Notlar:

- Ogrenci tarafinda sadece `active` durumdaki testler gorunur.
- `requires_student_account` MVP'de `false` olarak kullanilir. Bu, ogrencinin uyelik acmadan test cozebilecegi anlamina gelir; e-posta OTP ve KVKK onayi yine zorunludur.
- Sure zorunlu olmayabilir; `duration_minutes` bos ise suresiz test anlamina gelir.
- Public tarafta testin gorunebilmesi icin `status = active` olmalidir.
- `status = active` olan testlerde `starts_at` ve `ends_at` doluysa tarih araligi da kontrol edilir.
- `status = draft` olan test, tarih araligi uygun olsa bile public tarafta gorunmez.
- `ends_at` gecmisse test public tarafta gorunmez ve server-side sorgularda pasif sayilir.
- Bu kontrol, testin sorgulandigi anda server-side yapilacaktir (cron job yerine lazy evaluation).

7.7 test_questions

Test ve soru iliskisini tutar.

Alanlar:

- id
- test_id
- question_id
- order_index
- points (varsayilan: 1)
- created_at

Notlar:

- MVP'de her soru 1 puan olarak hesaplanır.
- Ileri fazda farkli puan vermek icin `points` alani teste gore degistirilebilir.

7.8 test_attempts

Ogrencinin test girisini tutar.

Alanlar:

- id
- test_id
- student_id
- status: `in_progress`, `completed`, `abandoned`
- started_at
- completed_at
- duration_seconds
- score
- correct_count
- wrong_count
- empty_count
- ip_address
- user_agent
- kvkk_accepted_at
- privacy_accepted_at
- terms_accepted_at
- created_at

Notlar:

- Ogrenci testi baslattiginda `in_progress` kaydi acilir.
- Test bitince skor ve sayim alanlari doldurulur.
- Ileri fazda yarim kalan testleri tamamlama veya auto-save bu tablo uzerinden gelistirilebilir.
- Ayni ogrencinin ayni testi tekrar cozmesini engellemek icin `(test_id, student_id)` unique index uygulanir.
- KVKK, gizlilik ve kullanim kosullari onay zamanlari attempt uzerinde saklanir.
- Sure doldugunda server mevcut attempt'i otomatik tamamlar; o ana kadar gonderilmis cevaplar uzerinden sonuc hesaplanir, gonderilmemis sorular bos sayilir.
- Sayfa yenilenirse veya baglanti koparsa MVP'de cevaplar otomatik kaydedilmez. Auto-save ileri faza birakilmistir.
- Puanlama MVP'de yuzdelik skor olarak hesaplanir: $\text{score} = \text{correct_count} / \text{total_question_count} * 100$.
- Yanlis cevaplar dogru cevaplari goturmez.

7.9 student_answers

Ogrenci cevaplarini tutar.

Alanlar:

- id
- attempt_id

- question_id
- selected_option (nullable)
- is_correct
- answered_at

Notlar:

- Bos birakilan sorularda selected_option null olarak kaydedilir; is_correct false olur.
- Sonuc detay sayfasi bu tabloyu kullanir.
- Test bittiginde tum cevaplar tek bir veritabani transaction'i icinde batch insert ile kaydedilir. Bu hem performansi arttirir hem de yarim kalan kayitlari onler.

7.10 ads

Reklam bolumlerini tutar. Ilk fazda statik reklam alanlari kullanilabilir; dinamik reklam yonetimi ilerleyen fazda eklenir.

Alanlar:

- id
- title
- description
- image_url
- target_url
- placement: home , course_list , result
- is_active
- starts_at
- ends_at
- created_at
- updated_at

7.11 Sistem Ayarlari

Ilk faz icin settings tablosu eklenebilir.

Kullanilabilecek alanlar:

- id
- key
- value
- updated_by
- updated_at

Kullanim alanlari:

- Site adi
- Varsayilan test suresi
- Reklam ayarlari
- Bakim modu

7.12 Yetkilendirme ve Veri Erisimi Taslagi

Veritabani seviyesinde RLS kullanilmadigi icin tum veri erisimi Next.js server action, route handler ve servis fonksiyonlari icinde kontrol edilir. Client componentler dogrudan veritabani sorgusu calistirmaz.

Veri erisim kurallari:

- profiles : Kullanici kendi profilini gorebilir. Admin tum profilleri yonetebilir.
- students : Admin tum ogrencileri gorebilir. Hoca, kendisine tanimli ogrencileri ve kendi testlerine girmis ogrencileri gorebilir.
- teacher_students : Admin tum eslestirmeleri yonetebilir. Hoca kendi eslestirmelerini okuyabilir.
- questions : Hoca kendi sorularini yonetebilir. Admin tum sorulari gorebilir.
- tests : Hoca kendi testlerini yonetebilir. Public tarafta sadece aktif testler okunabilir.
- test_attempts : Hoca kendi testlerine ait denemeleri gorebilir. Admin tum denemeleri gorebilir.
- student_answers : Hoca kendi testlerine ait cevaplari gorebilir. Admin tum cevaplari gorebilir.
- ads : Public tarafta aktif reklamlar okunabilir. Admin reklam yonetimi yapabilir.

Hoca ogrenci gorunurlugu karari:

- Hoca, kendi olusturdugu testlere giren tum ogrencilerin sonuc kayitlarini gorebilir.
- Hoca, admin tarafından kendisine tanımlanan ogrencileri `teacher_students` üzerinden ayrıca gorebilir.
- Hoca, baska hocalarin testlerine ait sonuc ve cevap detaylarini goremez.

Teknik uygulama:

- `requireTeacher()` aktif session kullanicisinin hoca oldugunu kontrol eder.
- `requireAdmin()` aktif session kullanicisinin admin oldugunu kontrol eder.
- `getTeacherProfile()` session kullanicisinden hoca profilini getirir.
- `assertOwnsQuestion(questionId)` sorunun aktif hocaya ait oldugunu dogrular.
- `assertOwnsTest(testId)` testin aktif hocaya ait oldugunu dogrular.
- `assertCanViewAttempt(attemptId)` hocanın ilgili test sonucunu gorebilecegini dogrular.

7.13 MVP Disi Ama Planlanan Veri Ozellikleri

İleri fazlarda eklenebilecek veri ozellikleri:

- Coklu soru tipi
- Sik karistirma
- Soru sirasi karistirma
- PDF sonuc raporu
- Ogrenci uyeligi
- Reklam tiklanma istatistikleri
- Hoca bazli detayli analitikler
- Degisiklik izleme (audit trail)

7.14 Degisiklik Izleme (İleri Faz)

İleri fazda `audit_logs` tablosu eklenecektir:

- id
- user_id
- action: `create`, `update`, `delete`
- entity_type: `question`, `test`, `student` vb.
- entity_id
- old_values (JSON)
- new_values (JSON)
- created_at

Bu tablo sayesinde hoca bir soruyu degistirdiginde eski hali kaybolmaz ve admin tarafından denetlenebilir.

8. Fazlara Bolunmus Yol Haritasi

Faz 0: Proje Hazirligi

Durum: Buyuk olcude tamamlandi. PRD ve mimari kararlar bu dokumanda yer alır.

Amac: Projenin temel kurallarini, teknoloji secimini ve is bolumunu netlestirmek.

Tamamlanan isler:

- PRD ilk surumu olusturuldu.
- Teknoloji stack'i secildi (bkz. Bolum 6).
- Veritabani modeli taslagi cikarildi (bkz. Bolum 7).
- Is bolumu netlestirildi (bkz. Bolum 9 ve 10).

Bu fazda kalan isler:

- Git repo kurulumu ve branch duzeni.
- Tasarim dili icin ortak referans (renk paleti, font, ikon seti).
- Rakip/benzer site referans inceleme.

Sorumluluk:

- Arda: Repo kurulumu, branch ve commit kurallari.
- Emir: Tasarim dili, renk/font referansi, rakip site incelemesi.

Kabul kriterleri:

- Repo hazir ve iki kisi de erisebiliyor.
- Tasarim dili icin ortak bir referans belirlenmis.
- Faz 1'e baslamak icin tum kararlar netlesmis.

Faz 1: Proje Iskeleti ve Ana Sayfa

Amac: Uygulamanin temel iskeletini ve ilk gorunen yuzunu olusturmak.

Isler:

- Proje kurulumu.
- Sayfa routing yapisi.
- Ortak layout.
- Header ve footer.
- Ana sayfa.
- Hoca giris butonu.
- Online test butonu.
- Statik reklam alani.

Sorumluluk:

- Arda: Proje kurulumu, routing, layout yapisi.
- Emir: Ana sayfa tasarimi, header/footer, reklam alani.

Kabul kriterleri:

- Ana sayfa acilir.
- Online test butonu ders secim sayfasina gider.
- Hoca giris butonu giris sayfasina gider.
- Reklam alani gorunur.

Faz 2: Dersler ve Ogrenci Test Akisi

Amac: Ogrencinin ders secip teste baslayabildigi temel akisi kurmak.

Isler:

- Ders modeli.
- Ders listeleme sayfasi.
- Test baslangic sayfasi.
- Ogrenci bilgi formu.
- Test oturumu baslatma.

Sorumluluk:

- Arda: Ders verisi, test oturumu mantigi, form validasyonu.
- Emir: Ders kartlari, ogrenci bilgi formu UI, bos durum tasarimlari.

Kabul kriterleri:

- Ogrenci dersleri gorebilir.
- Bir derse basinca test baslangic sayfasina gider.
- Ogrenci bilgilerini girerek teste baslayabilir.

Faz 3: Test Cozme ve Sonuc Hesaplama

Amac: Ogrencinin testi tamamlayip sonucunu gorebilmesini saglamak.

Isler:

- Soru gosterme ekrani.

- Sik secme.
- Cevaplari gecici olarak tutma.
- Testi bitirme.
- Dogru/yanlis/bos hesaplama.
- Sonuc ekrani.
- Sonucu veritabanina kaydetme.

Sorumluluk:

- Arda: Test motoru, cevap hesaplama, veritabani kaydi.
- Emir: Soru karti UI, sonuc ekrani tasarimi.

Kabul kriterleri:

- Ogrenci sorulari cevaplayabilir.
- Testi bitirebilir.
- Sonucunu gorebilir.
- Sonuc sistemde kayitli kalir.

Faz 4: Hoca Girisi ve Panel

Amac: Hocalarin guvenli sekilde panele girmesini saglamak.

Isler:

- Hoca giris sayfasi.
- Auth.js entegrasyonu ve session yonetimi.
- Rol kontrolu (admin ve teacher).
- Korumali route ve middleware.
- Hoca panel layout'u.
- Panel ana sayfasi.
- Admin tarafından ilk hoca hesaplarinin manuel olarak olusturulmasi.

Sorumluluk:

- Arda: Auth, rol kontrolu, korumali sayfalar.
- Emir: Giris formu UI, panel layout gorsel duzen.

Kabul kriterleri:

- Hoca giris yapabilir.
- Giris yapmayan kisi hoca paneline giremez.
- Hoca panel ana sayfasini gorur.

Faz 5: Soru Havuzu

Amac: Hocalarin soru yukleyebilmesini ve havuzdan soru gorebilmesini saglamak.

Isler:

- Soru ekleme formu.
- Soru listeleme.
- Ders filtresi.
- Zorluk filtresi.
- Soru duzenleme.
- Soru pasife alma.

Sorumluluk:

- Arda: Soru CRUD, yetki kontrolu, veritabani islemleri.
- Emir: Soru liste UI, form alanlari, filtre arayuzu.

Kabul kriterleri:

- Hoca soru ekleyebilir.
- Soru havuzunda sorular listelenir.
- Hoca sorulari derse gore filtreleyebilir.

- Hoca soru duzenleyebilir veya pasife alabilir.

Faz 6: Test Olusturma

Amac: Hocalarin soru havuzundan test olusturabilmesini saglamak.

Isler:

- Test olusturma formu.
- Ders secimi.
- Havuzdan soru secimi.
- Test soru sirasi.
- Test aktif/pasif ayari.

Sorumluluk:

- Arda: Test olusturma mantigi, test_questions iliskisi, aktif test sorgulari.
- Emir: Test formu UI, soru secim listesinin gorsel duzeni.

Kabul kriterleri:

- Hoca yeni test olusturabilir.
- Teste soru ekleyebilir.
- Testi aktif yapabilir.
- Aktif test ogrenci tarafinda gorunur.

Faz 7: Ogrenci ve Sonuc Takibi

Amac: Hocalarin sinava giren ogrencileri ve sonuclari takip etmesini saglamak.

Isler:

- Sinava giren ogrenciler listesi.
- Ogrenci detay sayfasi.
- Test sonucu detaylari.
- Kendisine tanimli ogrenciler listesi.
- Admin tarafından hoca-ogrenci eslestirme mantigi.

Sorumluluk:

- Arda: Sorgular, yetki kontrolu, hoca-ogrenci eslestirme, detay sayfalari.
- Emir: Liste ve tablo tasarımlari, bos durumlar ve filtre alanlari.

Kabul kriterleri:

- Hoca sinava giren ogrencileri gorebilir.
- Hoca ogrenci sonuc detayini gorebilir.
- Hoca kendisine tanimli ogrencileri gorebilir.
- Hoca yetkisi olmayan veriye erisemez.

Faz 8: Reklam Alanlari

Amac: Reklam bolumlerini kullanilabilir hale getirmek.

Isler:

- Ana sayfa reklam alanı.
- Ders sayfasi reklam alanı.
- Sonuc sayfasi reklam alanı.
- Ileri fazda admin panelden reklam yonetimi.

Sorumluluk:

- Emir: Statik reklam alanlari, gorsel duzen, placeholder tasarımlar.
- Arda: Dinamik reklam modeli ve admin yonetimi, ileri faz.

Kabul kriterleri:

- Reklam alanlari sayfalarda gorunur.

- Reklam alanlari tasarimi bozmaz.
- Ileri faz icin veri modeli hazirdir.

Faz 9: Test, Temizlik ve Yayina Hazirlik

Amac: Projeyi kullanilabilir ve sunulabilir hale getirmek.

Isler:

- Kritik akislari test etmek.
- Mobil uyumluluk kontrolu.
- Form validasyonlarini kontrol etmek.
- Hata mesajlarini duzenlemek.
- Seed/demo verileri hazirlamak.
- Deployment.

Sorumluluk:

- Arda: Teknik testler, deployment, veri guvenligi, performans kontrolu.
- Emir: Mobil tasarim kontrolu, metin duzeltmeleri, demo veri girisi, UI hatalarini raporlama.

Kabul kriterleri:

- Ogrenci testi bastan sona cozebilir.
- Hoca soru ve test yonetebilir.
- Hoca sonuclari gorebilir.
- Site mobilde kullanilabilir.
- Demo yayin linki hazirdir.

9. Arda Icin Net Gorev Sahipligi

Arda asagidaki modullerin tamamlanmasindan sorumludur:

9.1 Proje Altyapisi

- Proje kurulumunu yapmak.
- Klasor yapisini belirlemek.
- Routing yapisini kurmak.
- Ortak layout mantigini hazirlamak.
- Ortam degiskenleri ve proje ayarlarini duzenlemek.

9.2 Veritabani ve Veri Modeli

- Veritabani tablolarini tasarlamak.
- `profiles`, `students`, `teacher_students`, `courses`, `questions`, `tests`, `test_questions`, `test_attempts`, `student_answers` tablolarini olusturmak.
- Tablo iliskilerini ve gerekli indeksleri planlamak.
- Demo veri veya seed yapisini hazirlamak.

9.3 Auth ve Yetkilendirme

- Hoca giris sistemini kurmak.
- Admin ve teacher rollerini ayirmak.
- Hoca panelini giris yapmayan kullanicilara kapatmak.
- Hocalarin yalnızca yetkili olduklari verilere erismesini saglamak.

9.4 Ogrenci Test Akisi

- Ogrenci bilgi formundan gelen veriyi dogrulamak.
- Test oturumu baslatma mantigini yazmak.
- Sorularin ogrenciye dogru sekilde gelmesini saglamak.

- Cevaplari kaydetmek.
- Test bitince sonucu hesaplamak.
- Sonucu veritabanina kaydetmek.

9.5 Soru Havuzu ve Test Olusturma

- Soru ekleme, duzenleme, listeleme ve pasife alma islemlerini yazmak.
- Soru havuzunu ders ve zorluk bilgisine gore sorgulanabilir hale getirmek.
- Test olusturma mantigini yazmak.
- Test ile soru iliskisini kurmak.
- Aktif/pasif test durumunu yönetmek.

9.6 Hoca Paneli Veri Akislari

- Dashboard icin gerekli sayisal verileri hazirlamak.
- Sinava giren ogrenciler listesini olusturmak.
- Ogrenci sonuc detaylarini getirmek.
- Kendisine tanimli ogrenciler akisini kurmak.
- Hoca-ogrenci eslestirme mantigini hazirlamak.

9.7 Admin Paneli Veri Akislari

- Admin paneli icin korumali rotalari hazirlamak.
- Hoca hesabi olusturma, duzenleme ve pasife alma akisini yazmak.
- Ders olusturma ve duzenleme akisini yazmak.
- Hoca-ogrenci eslestirme yonetimini yazmak.
- Reklam yonetimi backend mantigini ileri fazda hazirlamak.

9.8 Yayina Alma ve Kontrol

- Kritik akislari test etmek.
- Deployment ayarlarini yapmak.
- Ortam degiskenlerini yayina hazirlamak.
- Son teknik kontrol ve kod review yapmak.

10. Emir Icin Net Gorev Sahipligi

Emir asagidaki modullerin tamamlanmasindan sorumludur:

10.1 Ana Sayfa ve Ortak Arayuz

- Ana sayfa tasarimini yapmak.
- Header componentini hazirlamak.
- Footer componentini hazirlamak.
- "Online Teste Basla" ve "Hoca Girişi" yonlendirme butonlarini arayuzde konumlandirmek.
- Ana sayfadaki tanitim metinlerini ve gorsel bolumleri hazirlamak.

10.2 Ders Secim Arayuzu

- Ders karti componentini hazirlamak.
- Ders listeleme sayfasinin gorsel duzenini yapmak.
- Aktif test olmayan dersler icin bos durum tasarimini hazirlamak.
- Ders kartlarinin mobil gorunumunu kontrol etmek.

10.3 Ogrenci Test Arayuzleri

- Test baslangic sayfasi arayuzunu hazirlamak.

- Öğrenci bilgi formu alanlarını tasarlamak.
- Test kuralları bölümünü hazırlamak.
- Soru kartı görsel düzenini yapmak.
- Sıkların seçilebilir görünümünü hazırlamak.
- Önceki, sonraki ve testi bitir butonlarının arayüzünü hazırlamak.
- Sonuç ekranı tasarımını yapmak.

10.4 Hoca Paneli Arayüzleri

- Hoca giriş formu arayüzünü hazırlamak.
- Hoca panel layout'unun görsel düzenini yapmak.
- Dashboard kartlarını tasarlamak.
- Soru havuzu liste ekranının arayüzünü hazırlamak.
- Soru ekleme/düzenleme formunun arayüzünü hazırlamak.
- Test oluşturma formunun arayüzünü hazırlamak.
- Sınava giren öğrenciler liste/tablo görünümünü hazırlamak.
- Kendisine tanımlı öğrenciler liste görünümünü hazırlamak.

10.5 Admin Paneli Arayüzleri

- Admin panel layout'unu hazırlamak.
- Hoca yönetimi liste ve form arayüzünü hazırlamak.
- Ders yönetimi liste ve form arayüzünü hazırlamak.
- Hoca-öğrenci eşleştirme arayüzünü hazırlamak.
- Reklam yönetimi arayüzünü ileri fazda hazırlamak.

10.6 Reklam ve İçerik Alanları

- Ana sayfa reklam alanını hazırlamak.
- Ders seçim sayfası reklam alanını hazırlamak.
- Sonuç sayfası reklam alanını hazırlamak.
- Reklam alanları için placeholder metin ve görselleri hazırlamak.
- Sayfa metinlerini ve demo içerikleri düzenlemek.

10.7 Responsive ve Görsel Kontrol

- Ana sayfa mobil görünümünü kontrol etmek.
- Ders seçim ve test ekranlarının mobil görünümünü kontrol etmek.
- Hoca panelindeki liste ve kartların responsive davranışını kontrol etmek.
- Bos durum, hata durumu ve yükleniyor durumları için arayüz hazırlamak.

10.8 Ortak Teslim Kuralları

- Her görevin sonunda ilgili ekran çalışır halde teslim edilir.
- Arda, Emir'in hazırladığı arayüzleri veri ve iş mantığına bağlar.
- Emir, Arda'nın hazırladığı veri akışları için gerekli arayüz düzenlemelerini tamamlar.
- Faz sonunda iki kişi birlikte öğrenci ve hoca akışlarını kontrol eder.

11. Önerilen Haftalık Çalışma Planı

Haftalık plan 6 haftaya yayılmıştır ve Bölüm 8'deki fazlarla eşleşerek ilerler.

Hafta 1: Faz 0 ve Faz 1 (Hazırlık + İskelet)

Hedef:

- Repo kurulumu, branch ve commit düzeni
- Tasarım dili referansı

- Next.js + TypeScript + Tailwind + shadcn/ui kurulumu
- Ortak layout, header, footer
- Ana sayfa ilk hali

Arda:

- Repo kurulumu, Next.js + Prisma + Auth.js iskeleti.
- Route gruplari ve layout yapisi.
- `.env` ve Vercel ortam degisken duzeni.

Emir:

- Tasarim dili referansi (renk, font).
- Ana sayfa tasarimi.
- Header ve footer componenti.
- Statik reklam alani placeholder'i.

Hafta 2: Faz 2 (Dersler ve Ogrenci Akisi)

Hedef:

- Ders modeli ve listeleme sayfasi
- Test baslangic sayfasi
- Ogrenci bilgi formu

Arda:

- Prisma migration ile `courses`, `students`, `tests`, `test_questions` tablolari.
- Test oturumu baslatma server action.
- Form validasyonu (Zod).

Emir:

- Ders kartlari ve liste sayfasi UI.
- Ogrenci bilgi formu UI.
- Bos durum ekranlari.

Hafta 3: Faz 3 (Test Cozme ve Sonuc)

Hedef:

- Test cozme ekrani
- Cevap kaydi ve sonuc hesaplama
- Sonuc ekrani

Arda:

- `test_attempts` ve `student_answers` migration ve servis fonksiyonlari.
- Sonuc hesaplama (dogru/yanlis/bos/puan).
- Server-side sure kontrolu.

Emir:

- Soru karti UI.
- Sik secimi ve gezinme.
- Sonuc ekrani tasarimi.

Hafta 4: Faz 4 ve Faz 5 (Hoca Girisi + Soru Havuzu)

Hedef:

- Auth.js entegrasyonu
- Hoca panel layout'u
- Soru havuzu CRUD

Arda:

- Auth.js Prisma adaptoru ve session yonetimi.

- Korumali rota ve middleware.
- `questions` CRUD ve yetki kontrolu.

Emir:

- Giriş formu UI.
- Hoca panel layout'u (sidebar + topbar).
- Dashboard kartları.
- Soru havuzu liste ve form UI.

Hafta 5: Faz 6 ve Faz 7 (Test Oluşturma + Sonuç Takibi)

Hedef:

- Test oluşturma akisi
- Sınava giren öğrenciler
- Hoca-öğrenci eşleştirme
- Admin paneli ilk hali

Arda:

- Test oluşturma servisi ve `test_questions` ilişkisi.
- Hoca-öğrenci eşleştirme ve admin servisleri.
- Sınava giren öğrenciler sorguları.

Emir:

- Test oluşturma formu UI.
- Sınava giren öğrenciler tablosu.
- Tanımlı öğrenciler liste UI.
- Admin paneli liste arayüzleri.

Hafta 6: Faz 8 ve Faz 9 (Reklam + Yayına Hazırlık)

Hedef:

- Reklam alanları
- KVKK ve içerik sayfaları
- Test güvenliği gözden geçirme
- Mobil ve görsel son kontroller
- Deployment

Arda:

- Test güvenliği kontrolleri (yeniden giriş, süre, server-side doğrulama).
- KVKK ve içerik sayfaları için route hazırlama.
- Vercel production deployment.
- Hata yönetimi ve loglama gözden geçirme.

Emir:

- Reklam alanlarının tamamlanması.
- İletişim, Gizlilik, KVKK metni ve Kullanım Kosulları sayfaları.
- Mobil görünüm kontrolleri.
- Erişilebilirlik ve içerik son okuması.

12. MVP Kapsamı (Sayfa ve Akis Bazında)

Bu bölüm, MVP'de yer alacak sayfaları ve akisleri listeler. Özellikle bazında kapsam için Bölüm 6.12'ye bakılmalıdır.

İlk çalışan sürümde olacak sayfalar ve akisler:

- Ana sayfa
- "Online Test"e yönlendirme
- Ders listeleme sayfası

- Test baslangic sayfasi (ogrenci bilgi formu + test kurallari + KVKK onay)
- Test cozme ekranı
- Sonuc ekranı (ozet + cevap detayı)
- Hoca giris sayfasi
- Hoca paneli ana sayfasi
- Soru havuzu sayfasi (liste + ekle/duzenle)
- Test yonetimi sayfasi (liste + ekle/duzenle)
- Sinava giren ogrenciler sayfasi
- Tanimli ogrenciler sayfasi
- Admin paneli (hoca ve ders yonetimi)
- Zorunlu icerik sayfaları (İletişim, Gizlilik Politikası, KVKK Aydınlatma Metni, Kullanım Kosulları)
- Statik reklam alanları (ana sayfa, ders listesi, sonuc)

İlk surumde olmayacak sayfalar ve akislar:

- Ogrenci uyeligı ve oturum yonetimi
- Odeme sayfasi
- Detayli istatistik/grafik sayfaları
- Sertifika sayfasi
- Coklu dil destegi
- Hakkimizda ve SSS sayfaları
- SMS ve bildirim ayarları
- Yapay zeka ile soru uretimi
- Reklam yonetim paneli (admin tarafında dinamik reklam yonetimi)

13. Verilmis Kararlar

PRD'nin tek karar kaynagi olması prensibiyle, projenin ilerleyisini etkileyen kararlar bu bolumde net olarak yazilmistir. Bir karar degistirilmek istendiginde once bu bolum guncellenir, sonra kod yazilir.

13.1 Ogrenci Bilgi Toplama

- Ogrenci, teste baslamadan once asagidaki bilgileri girer:
- Ad-soyad (zorunlu)
- E-posta (zorunlu)
- Telefon (opsiyonel)
- Sinif seviyesi (zorunlu)
- Bagli oldugu hoca (opsiyonel, dropdown)
- Ogrenci girisi/uyeligı MVP'de yoktur. Tekrar test cozme kontrolu dogrulanmis e-posta uzerinden yapilir.
- Test baslatma akisi sirayla su sekildedir:
- Ogrenci test baslangic sayfasında bilgilerini girer.
- KVKK, gizlilik ve kullanım kosullari onaylarını verir.
- E-posta adresine OTP gonderilir.
- OTP dogrulaniir.
- `students` tablosunda e-posta ile upsert yapilir.
- `(test_id, student_id)` unique kontroluyle tekrar deneme engellenir.
- `test_attempts` kaydi transaction icinde `in_progress` olarak olusturulur.
- Ogrenci test cozme ekranina yonlendirilir.

13.2 Hoca Hesabi Yonetimi

- Hocalar kendileri kayıt olamaz.
- Tum hoca hesapları admin tarafından olusturulur.
- Public bir kayıt sayfasi MVP'de yoktur.

13.3 Aktif Test Sayisi

- Bir ders icinde ayni anda birden fazla aktif test olabilir.
- Ogrenci bir dersi sectiginde aktif testlerin listesini gorur.
- Ogrenci listeden bir test sectiginde `/test/[testId]/start` sayfasina gider.

13.4 Soru Havuzu Sahipligi

- Sorular hoca bazlidir; her hoca yalnızca kendi sorularini gorur ve teste ekler.
- Ortak soru havuzu MVP'de yoktur, ileri faza birakilmistir.

13.5 Reklam Modeli

- Reklamlar MVP'de statiktir.
- Admin panelinden dinamik reklam yonetimi ileri faza birakilmistir.
- Reklam ag entegrasyonu (AdSense vb.) MVP'de yoktur.

13.6 Soru Silme

- Sorular kalici olarak silinmez.
- `is_active = false` ile pasife alinir (soft-delete).
- Bir teste ekli sorular pasife alinsa bile o testlerde gorunmeye devam eder.

13.7 Tekrar Deneme

- Ayni testin ayni ogrenci tarafından tekrar tekrar cozulmesi MVP'de engellenir.
- Tekil deneme kontrolu `test_attempts` tablosunda (`test_id`, `student_id`) uzerinden unique index ile saglanir.
- Ogrenci kimligi OTP ile dogrulanmis e-posta uzerinden belirlenir (bkz. Bolum 4.1 Kimlik dogrulama).
- OTP dogrulamasindan gecmis ogrenci kaydi `students` tablosundaki mevcut kayitla eslestirilir.
- "Tekrar denemeye izin ver" ayari ileri faza birakilmistir.

13.8 Sure Yonetimi

- Test bazinda `duration_minutes` belirlenebilir.
- Bos birakilirsa test suresizdir.
- Sure server-side kontrol edilir ve dolunca cevap kabul edilmez.

13.9 Dil

- Tek dil: Turkce.
- Coklu dil destegi MVP'de yoktur.

13.10 Yasal

- Aydinlatma metni ve KVKK onay kutusu test baslamadan once gosterilir.
- Gizlilik politikasi, KVKK aydinlatma metni ve kullanim kosullari sayfalari yayina alinmadan once hazir olur.

13.11 Esanli Duzenleme

- Iki hocanın ayni anda ayni soruyu veya testi duzenlemesi MVP'de engellenmez.
- Son kaydi yapan kisinin degisiklikleri gecerli olur (last-write-wins).
- Ileri fazda `updated_at` alanı uzerinden optimistic concurrency control ("Bu kayit baskasi tarafından degistirildi" uyarisi) eklenebilir.

13.12 E-posta Servisi

- OTP gondermek, sifre sifirlama ve bildirimler icin Resend kullanılacaktır.
- Resend, Vercel ile uyumlu ve ucretsiz katmanda gunluk 100 e-posta destegi sunar.
- `RESEND_API_KEY` environment variable olarak saklanacaktır.
- E-posta sablonlari React Email ile olusturulacaktır.

- SMS entegrasyonu (Twilio, Netgsm vb.) ileri faza bırakılmıştır.

13.13 URL ve Route Dili

- MVP'de route isimleri teknik olarak kısa ve tutarlı tutulur.
- Public test akisi için ana route `/online-test` olarak kalır.
- SEO için ileride Türkçe alias route'lar (`/dersler` gibi) eklenebilir, ancak MVP'de tek route seti kullanılır.

13.14 Branch ve Commit Kuralları

- Ana branch `main` olur.
- Aktif geliştirme için `dev` branch'i kullanılır.
- Yeni işler `feature/kısa-acıklama`, hata düzeltmeleri `fix/kısa-acıklama` branch'lerinde yapılır.
- Commit mesajları kısa ve emir kipinde yazılır: `add teacher login`, `fix test scoring` gibi.

13.15 Seed ve Demo Veri

MVP seed verisi asgari olarak şunları içerir:

- 1 admin kullanıcısı
- 1 hoca kullanıcısı
- 3 ders
- Her ders için en az 5 soru
- En az 1 aktif test
- En az 1 tamamlanmış test denemesi ve sonuç kaydı

14. Başarı Kriterleri

Proje başarılı sayılacaksa:

- Öğrenci üye olmadan test çözebilmeli.
- Test sonucu doğru hesaplanmalı ve kaydedilmeli.
- Hoca giriş yapıp panelini görebilmeli.
- Hoca soru ekleyip test oluşturabilmeli.
- Hoca sınava giren öğrencileri görebilmeli.
- Hoca kendisine tanımlı öğrencileri görebilmeli.
- Sistem mobilde temel olarak kullanılabilir olmalı.
- Reklam alanları sayfalarda yer almalı.

15. Dosya ve Kod Organizasyonu Kuralları

Proje kodlanırken:

- Her dosya mümkünse 500 satırı geçmemeli.
- Sayfalar, componentler, servisler ve veritabanı işlemleri ayrılmalı.
- Tekrarlanan UI parçaları component haline getirilmeli.
- İş mantığı componentlerin içine dağıtılmamalı.
- Kritik kararlar önce bu PRD dosyasına eklenmeli.
- Fazla işler ayrı not edilmeli ve plan dışına çıkılmamalı.

Önerilen klasör mantığı:

- `src/app` : Next.js App Router sayfaları ve route grupları
- `src/app/(public)` : Ana sayfa ve public ders/test giriş ekranları
- `src/app/(student)` : Öğrenci test akisi
- `src/app/(teacher)` : Hoca paneli
- `src/app/(admin)` : Admin paneli
- `src/components` : Ortak UI bileşenleri

- `src/components/ui` : shadcn/ui bileşenleri
- `src/features` : Öğrenci test akisi, hoca paneli, soru havuzu gibi modüller
- `src/lib` : Prisma client, auth yardımcıları, validasyon ve servis fonksiyonları
- `src/types` : Ortak TypeScript tipleri
- `src/styles` : Global stiller
- `prisma` : Prisma schema, migration ve seed dosyaları

Onerilen feature klasörleri:

- `src/features/student-test`
- `src/features/teacher-dashboard`
- `src/features/question-bank`
- `src/features/test-builder`
- `src/features/results`
- `src/features/ads`

16. İlk Yapılacaklar

1. Next.js + TypeScript + Tailwind CSS projesini kur.
2. shadcn/ui kurulumunu yap ve temel UI componentlerini ekle.
3. Prisma kurulumunu yap.
4. PostgreSQL için Neon üzerinde veritabanı oluştur.
5. Auth.js (NextAuth) Prisma adaptörü kurulumunu hazırla.
6. `profiles`, `students`, `otp_verifications`, `teacher_students`, `courses`, `questions`, `tests`, `test_questions`, `test_attempts`, `student_answers` tablolarını Prisma migration olarak hazırla.
7. Resend ve Upstash environment variable listesini oluştur.
8. Vercel deployment için environment variable listesini oluştur.
9. Ana sayfa ve route gruplarını oluştur.
10. Ders listeleme ve öğrenci test akisini başlat.
11. Her faz sonunda PRD'yi güncelleyip ilerlemeyi kontrol et.

17. Arastırma Kaynakları

Bu PRD güncellemesinde dikkate alınan kaynaklar:

- Online sınav sistemlerinde yaygın özellikler: soru havuzu, süreli test, otomatik sonuç, öğrenci cevap detayları, hoca/admin dashboard'u ve raporlama.
- Prisma + Auth.js + Next.js dokümanı: <https://prisma.io/docs/guides/authjs-nextjs>
- Next.js App Router proje yapısı dokümanı: <https://nextjs.org/docs/app/getting-started/project-structure>
- shadcn/ui dokümanı: <https://ui.shadcn.com/docs>
- Vercel environment variables dokümanı: <https://vercel.com/docs/environment-variables/manage-across-environments>
- Firebase Security Rules dokümanı: <https://firebase.google.com/docs/rules/rules-and-auth>
- Appwrite Auth dokümanı: <https://appwrite.io/docs/advanced/security/authentication>
- PocketBase production dokümanı: <https://pocketbase.io/docs/going-to-production>
- Laravel Sanctum dokümanı: <https://laravel.com/docs/11.x/sanctum>

Arastirmedan cikan ana sonuc:

- Bu proje için seçilen MVP yaklaşımı; Next.js App Router ile tek uygulama, Prisma ile PostgreSQL veritabanı erişimi, Auth.js (NextAuth) ile hoca/admin girişi, Tailwind + shadcn/ui ile arayüz ve Vercel ile deployment şeklindedir.
- Ayri backend uygulaması ilk sürüm için gerekli değildir.
- Yetki kontrolü sadece frontend kontrollerine bırakılmayacak, server-side servis fonksiyonları ve ownership kontrolleriyle uygulanacaktır.

18. Gizlilik, KVKK ve Hukuki Notlar

Bu bölüm, Türkiye'de yayına çıkacak bir online test platformu için minimum gerekli yasal ve gizlilik gerekliliklerini özetler.

18.1 Toplanacak Veriler

Sistem asagidaki kisisel verileri toplar:

- Ogrenci ad-soyad, e-posta, telefon, sinif/okul bilgisi
- Ogrenci test cevapları ve sonuc verileri
- `test_attempts` tablosunda `ip_address` ve `user_agent`
- Hoca ve admin hesap bilgileri (e-posta, ad-soyad)

18.2 KVKK Gereklilikleri

- Aydınlatma metni hazırlanacak ve "Teste Basla" sayfasında kullanıcıya gösterilecek.
- Öğrenci, kişisel verilerinin işlenmesini onaylamadan teste başlayamayacak.
- Kullanıcı verilerinin silinme talepleri için iletişim adresi yayınlanacak.
- Veriler sadece sistemin çalışması için gerekli süre boyunca saklanacak.
- Sağlık, dini inanc gibi özel nitelikli veri toplanmayacak.

Veri saklama ve silme:

- Öğrenci test sonuc verileri MVP'de 2 yıl saklanır.
- İletişim ve kimlik bilgileri, silme talebi gelmediği sürece sonuc verileriyle birlikte saklanır.
- Silme talebi geldiğinde öğrencinin ad-soyad, e-posta, telefon ve okul bilgisi anonimleştirilir.
- Test sonuc istatistikleri sistem raporlaması için anonim olarak tutulabilir.
- Hoca ve admin hesap kayıtları hesap aktif olduğu sürece saklanır; hesap kapatılınca pasife alınır.

18.3 Cerez ve KVKK Bildirimi

- Ana sayfada cerez bildirimi banner'i olacak.
- Sadece zorunlu cerezler kullanılacak; uclu parti analytics eklenirse onay alınacak.

18.4 Reklam Tarafı

- Reklamlar ilk fazda statik olacağı için reklam ağı'ları (Google AdSense vb.) MVP'de kullanılmayacak.
- Reklam ağı'ı eklenirse ek aydınlatma metni güncellemesi gerekecek.

19. Hata Yönetimi, Loglama ve İzleme

19.1 Hata Yönetimi

- Tüm server action ve route handler'lar try/catch ile sarıdır.
- Kullanıcıya teknik hata mesajı gösterilmez; "Bir sorun oluştu, lütfen tekrar deneyin." gibi sade Türkçe mesajlar gösterilir.
- Form validasyon hataları React Hook Form + Zod üzerinden alan bazlı gösterilir.
- Bilinmeyen route'lar için `not-found.tsx` sayfası olur.
- Beklenmeyen hatalar için `error.tsx` sayfası olur.

19.2 Loglama

- Gelistirme: Konsol logları yeterli.
- Production: Vercel Logs üzerinden temel inceleme yapılır.
- Kritik hatalar için Sentry entegrasyonu ileri fazda düşünülebilir.

Minimum production log formatı:

- `event` : hata veya işlem adı
- `request_id` : varsa istek takip id'si
- `user_id` : giriş yapmış hoca/admin varsa
- `route` veya `action` : hatanın oluştuğu yer
- `error_code` : kullanıcıya gösterilmeyen teknik hata kodu
- `created_at` : log zamanı

Loglanacak kritik olaylar:

- Basarisiz hoca giris denemeleri
- OTP gonderme ve OTP dogrulama hatalari
- Test baslatma hatalari
- Test tamamlama ve sonuc hesaplama hatalari
- Yetkisiz veri erisim denemeleri

19.3 Analytics

- Vercel Analytics ile sayfa görüntülemeleri ve performans izlenir.
- Reklam tıklamaları ileri fazda kendi tablomuza loglanabilir.

20. SEO ve Metadata

- Public sayfalarda Next.js metadata API kullanılarak `title`, `description` ve `og:image` belirlenir.
- Site genel `sitemap.xml` ve `robots.txt` dosyaları oluşturulur.
- Public sayfa URL'leri Türkçe ve okunabilir olacaktır (örneğin `/online-test`, `/dersler`).
- Hoca paneli ve admin sayfaları `noindex` olarak işaretlenir.

21. Erisilebilirlik ve Tarayıcı Destegi

- Hedef erişilebilirlik seviyesi: WCAG 2.1 AA (mumkun olan oranda).
- shadcn/ui Radix tabanlı olduğu için temel klavye ve ekran okuyucu desteği gelir.
- Renk kontrastı minimum 4.5:1 olacaktır.
- Tüm form elemanlarında `label` ve `aria-*` özellikleri doğru kullanılacaktır.
- Desteklenen tarayıcılar: Chrome, Edge, Firefox ve Safari'nin son 2 sürümü.
- Hedef ekran: Mobil 360px genişlikten yukarı.

Responsive breakpoint'ler:

- Mobil: 360px - 639px
- Tablet: 640px - 1023px
- Desktop: 1024px ve üzeri
- Büyük ekran: 1280px ve üzeri
- Public öğrenci ekranlarında mobil-first yaklaşım uygulanacaktır.
- Hoca panelinde sidebar 1024px altında collapse olacak veya hamburger menu'ye dönüşecektir.

22. Test Güvenliği

Online sınav platformlarında kritik bir noktadır.

22.1 Yeniden Giriş Kontrolü

- Bir testin aynı öğrenci tarafından tekrar tekrar çözülmesini engellemek için doğrulanmış e-posta bazlı tekil deneme kontrolü yapılır.
- `test_attempts` tablosunda (`test_id`, `student_id`) için tekil indeks veya policy uygulanır.
- Hoca dilerse "tekrar denemeye izin ver" ayarını ileri fazda acabilir.

22.2 Cevap Manipulasyonu

- Soruların doğru cevabı client'a gönderilmez.
- Sonuç hesaplaması sadece server-side yapılır.
- Cevaplar server action'a gönderildiğinde sunucu tarafında doğrulanır.

22.3 Test Süresi

- Sure varsa başladığı an `started_at` server-side kaydedilir.

- Sure dolunca server `completed_at` 'i otomatik isaretler ve cevap kabul etmez.
- Client tarafındaki geri sayim sayaci, her cevap gonderiminde server'dan kalan sureyi alarak kendini senkronize eder.
- Internet baglantisi kesildiginde client yerel zamanlayiciyi durdurmaz ancak baglanti donunce server'dan guncel sureyi alır.
- Client ve server arasindaki sure farki 5 saniyeyi asarsa client'taki sayac server suresine sifirlanir.
- Sure dolmadan 1 dakika kala kullaniciya gorsel ve sesli uyarı verilir.

22.4 Spam ve Bot Kontrolu

- Test baslatma endpoint'ine IP bazli rate limiting eklenir: ayni IP'den dakikada en fazla 5 test baslatma istegi.
- Hoca giris endpoint'ine rate limiting eklenir: ayni IP'den dakikada en fazla 10 giris denemesi.
- OTP gonderme endpoint'ine rate limiting eklenir: ayni e-posta adresine 5 dakikada en fazla 3 OTP istegi.
- Rate limiting MVP'de `@upstash/ratelimit` ve Upstash Redis ile uygulanacaktır.
- Gerekli environment variable'lar: `UPSTASH_REDIS_REST_URL` ve `UPSTASH_REDIS_REST_TOKEN`.
- Ileri fazda Cloudflare Turnstile veya hCaptcha ile bot korumasina gecilecektir.

23. Icerik Sayfalari

Public alanda olmasi gereken icerik sayfalari:

MVP'de zorunlu sayfalar:

- Iletisim
- Gizlilik Politikasi
- KVKK Aydınlatma Metni
- Kullanım Kosullari

Ileri faz sayfalari:

- Hakkımızda
- Sıkça Sorulan Sorular (SSS)

Sorumluluk:

- Emir: Sayfa yapisi, icerik metinlerinin yerlestirilmesi, tasarim.
- Arda: Routing ve metadata, gerektiğinde admin paneli üzerinden duzenlenebilir hale getirme (ileri faz).

24. Backend Alternatifleri (Arastirma Ozeti)

Bu bolum, ana karari vermeden once degerlendirilen backend seceneklerinin ozetidir. Secilen yapi her zaman Bolum 6'dadir.

24.1 Secilen Yapi: Next.js + Prisma + PostgreSQL

Bu proje icin ana yapi olarak secilmistir.

Kullanilacak yapi:

- Frontend: Next.js App Router
- Backend: Next.js server actions ve route handlers
- ORM: Prisma
- Veritabani: PostgreSQL
- Veritabani hosting: Neon
- Auth: Auth.js (NextAuth)
- Deployment: Vercel

Neden uygun:

- Proje iliskisel veri agirlikli oldugu icin PostgreSQL uygundur.
- Prisma migration ve type-safe sorgu destegi verir.
- Ayrica bir backend reposu gerektirmez.
- Iki kisilik ekip icin gelistirme hizi yuksektir.
- Vercel preview deployment akisiyle uyumludur.

Dikkat edilmesi gerekenler:

- Veritabani seviyesinde RLS olmadigi icin yetki kontrolleri server-side kodda dikkatli yazilmalidir.
- Neon connection pooling ayarlari Vercel serverless icin uygun yapilmalidir.
- Auth.js Prisma adaptoru ile `User` , `Account` , `Session` ve `VerificationToken` tablolari otomatik olusturulacaktır.

24.2 Firebase

Artlari:

- Auth, database, hosting ve storage tarafinda olgun servisler sunar.
- Hizli prototip cikarmak icin uygundur.
- Realtime ozellikleri gucludur.

Eksileri:

- Firestore NoSQL oldugu icin hoca, ogrenci, test, soru ve sonuc iliskileri PostgreSQL kadar rahat modellenmez.
- Maliyet buyudukce ongorulmesi zor olabilir.
- Vendor lock-in riski daha yuksektir.

Karar:

- Bu proje icin kullanilabilir ama ana tercih degildir.

24.3 Appwrite

Artlari:

- Auth, database, storage ve functions tek platformda gelir.
- Self-host edilebilir.
- Supabase benzeri bir backend deneyimi sunar.

Eksileri:

- Ekosistem ve kaynak sayisi Firebase/Supabase kadar genis degildir.
- Veritabani ve sorgu esnekligi PostgreSQL + Prisma kadar rahat olmayabilir.

Karar:

- Supabase benzeri hazır backend istenirse degerlendirilebilir.

24.4 PocketBase

Artlari:

- Tek binary ile cok hizli kurulur.
- Dahili admin paneli, auth ve SQLite veritabani vardir.
- Demo ve MVP icin cok pratiktir.

Eksileri:

- SQLite write concurrency nedeniyle buyuyen sistemlerde sinirli kalabilir.
- Yatay olceklemeye icin uygun degildir.
- Uzun vadeli ciddi urun icin risklidir.

Karar:

- Sadece demo veya cok kucuk MVP icin dusunulebilir.

24.5 NestJS + Prisma + PostgreSQL

Artlari:

- Ayri backend mimarisi icin profesyonel ve moduler bir secenektir.
- RBAC, servis katmani, API dokumantasyonu ve test yapisi gucludur.
- Buyuk projeye daha rahat evrilebilir.

Eksileri:

- Iki kisilik ekip icin kurulum ve gelistirme yuku artar.

- Ayrica backend deployment, API sozlesmeleri ve ekstra repo/klasor yonetimi gerekir.

Karar:

- İlk MVP icin gerekli degildir. Proje buyurse ileride ayrilmis backend olarak dusunulebilir.

24.6 Laravel + PostgreSQL

Artlari:

- Auth, policy, middleware ve admin panel ekosistemi gucludur.
- Laravel Sanctum ile API auth kolay kurulur.
- Spatie Permission gibi paketlerle rol/yetki sistemi hizli kurulabilir.

Eksileri:

- Next.js frontend ile birlikte iki ayri teknoloji ekosistemi yonetmek gerekir.
- Ekip PHP/Laravel tarafinda deneyimli degilse gelistirme hizi dusur.

Karar:

- Laravel bilen ekipler icin guclu bir secenektir; bu proje icin ana tercih degildir.

24.7 Genel Sira

Bu proje icin alternatif backend siralamasi:

1. Next.js + Prisma + PostgreSQL + Auth.js (NextAuth) - secilen yapi
2. Appwrite
3. Firebase
4. NestJS + Prisma + PostgreSQL
5. Laravel + PostgreSQL
6. PocketBase